

Loan Default Risk Prediction with Explainable AI

Deliverable 2: Model Architecture & Empirical Evaluation

23L-2524

23L-2560

Contents

1	Phase 3: Model Architecture & Implementation	2
1.1	Framework Selection	2
1.2	Pipeline Design	2
1.3	Feature Engineering	2
1.4	Model Comparison	2
1.5	Hyperparameters	3
1.6	Feature Selection	3
2	Phase 4: Training & Evaluation	4
2.1	Validation Strategy	4
2.2	Why These Metrics?	4
2.3	Threshold Tuning	4
2.4	Final Results	5
2.5	Risk Category Analysis	5
3	Error Analysis	7
3.1	Where Does the Model Fail?	7
3.2	Profile of Missed Defaults	7
4	Bias Check	8
4.1	Performance by Age Group	8
4.2	Performance by Income Group	8
4.3	Bias Conclusion	8
5	Contribution Table	9
6	Git Commit History	10

1 Phase 3: Model Architecture & Implementation

1.1 Framework Selection

The project uses **Scikit-Learn** and **XGBoost** as the primary frameworks. Scikit-Learn provides the preprocessing pipeline (RobustScaler, SMOTE via imbalanced-learn, train/test splitting), while XGBoost serves as the final model. XGBoost was chosen over alternatives because it handles tabular data well, provides built-in feature importance, and supports probability outputs needed for threshold tuning and risk categorization.

1.2 Pipeline Design

The full pipeline follows this order:

1. **Feature Engineering:** 3 new features added to the original 10
2. **Train/Validation/Test Split:** 70/15/15 stratified split
3. **Scaling:** RobustScaler fitted on training set only
4. **SMOTE:** Applied on training set only (ratio = 0.7)
5. **Train XGBoost:** With all 13 features
6. **Feature Selection:** Drop weak features based on importance
7. **Retrain:** With selected features only
8. **Threshold Tuning:** Optimize decision boundary on validation set
9. **Final Evaluation:** On held-out test set (touched once)

1.3 Feature Engineering

Three new features were derived from existing columns to create stronger signals:

Feature	Formula	Purpose
TotalLatePayments	$\text{Times30_59} + \text{Times60_89} + \text{Times90}$	Combines 3 weak signals into 1
HasSevereDelinquency	1 if $\text{Times90Late} > 0$, else 0	Clean binary flag for severe risk
IncomeDebtRatio	$\text{MonthlyIncome} / (\text{DebtRatio} + 1)$	Higher = safer borrower

Table 1: Engineered features

1.4 Model Comparison

Four models were compared on the validation set using the same preprocessing pipeline:

Model	Precision	Recall	F1	AUC-ROC
Logistic Regression	0.2969	0.5892	0.3948	0.8506
Random Forest	0.3898	0.3403	0.3634	0.8364
Gradient Boosting	0.3991	0.4723	0.4326	0.8589
XGBoost	0.4555	0.2914	0.3554	0.8500

Table 2: Model comparison on validation set

XGBoost was selected as the final model because it had the highest AUC-ROC and responds well to hyperparameter tuning and threshold adjustment.

1.5 Hyperparameters

Parameter	Values Tested	Final Selection
n_estimators	100, 200	200
max_depth	3, 5	5
learning_rate	0.05, 0.1	0.1
subsample	0.8	0.8
SMOTE ratio	0.5, 0.7, 1.0	0.7
Decision threshold	0.10 – 0.50	0.30

Table 3: Hyperparameter tuning

Justification: Higher `n_estimators` (200) gave the model more boosting rounds to learn. `max_depth=5` balanced complexity and overfitting. `learning_rate=0.1` provided good convergence speed. SMOTE ratio of 0.7 gave the model enough default examples without overwhelming the majority class.

1.6 Feature Selection

After training with all 13 features, feature importance was computed and models were retrained with different feature counts. The top 7 features by F1 were selected:

Feature	Importance
TotalLatePayments (engineered)	0.2042
RevolvingUtilization	0.2016
RealEstateLines	0.1613
HasSevereDelinquency (engineered)	0.1162
OpenCreditLines	0.1090
Times60_89Late	0.0943
Age	0.0496
<i>Dropped: Dependents, DebtRatio, MonthlyIncome, Times30_59Late, Times90Late, IncomeDebtRatio</i>	
< 0.03 each	

Table 4: Selected features ranked by importance

The engineered feature `TotalLatePayments` became the most important predictor, confirming that combining three weak late-payment signals into one produced a stronger feature than any individual column. `Times30_59Late` and `Times90Late` were dropped because their signal was absorbed by the engineered features.

2 Phase 4: Training & Evaluation

2.1 Validation Strategy

We use a **70/15/15 stratified Train/Validation/Test split**:

Split	Rows	No Default	Default
Training (70%)	104,463	97,460 (93.3%)	7,003 (6.7%)
Validation (15%)	22,385	20,884 (93.3%)	1,501 (6.7%)
Test (15%)	22,385	20,885 (93.3%)	1,500 (6.7%)

Table 5: Data split with stratified class distribution

- **Training set:** Used to train the model. SMOTE is applied only here.
- **Validation set:** Used for feature selection, threshold tuning, and model comparison. Touched multiple times during development.
- **Test set:** Touched only once at the very end for unbiased final evaluation.

5-Fold Stratified Cross-Validation was also run separately to confirm consistency across different data splits.

2.2 Why These Metrics?

Accuracy is misleading for this problem. With 93% non-defaulters, a model that predicts “No Default” for everyone achieves 93% accuracy while catching zero actual defaulters. We report these metrics instead:

- **Precision:** Of everyone flagged as “will default,” how many actually did? Low precision means too many false alarms.
- **Recall:** Of all actual defaulters, how many did the model catch? Low recall means missed defaults and bad loans getting approved.
- **F1-Score:** The harmonic mean of Precision and Recall. Only high when both are reasonable.
- **AUC-ROC:** Measures the model’s ranking ability across all thresholds. A high AUC means the model consistently assigns higher risk scores to actual defaulters.

For lending, **Recall matters most**: missing a defaulter (approving a bad loan) is far more costly than a false alarm (extra review of a good applicant). This is why we perform threshold tuning to boost Recall.

2.3 Threshold Tuning

By default, a probability > 0.5 is classified as Default. With 93/7 imbalance, very few applicants cross 0.5. Lowering the threshold catches more defaulters at the cost of more false alarms.

Threshold	Precision	Recall	F1	Caught
0.50 (default)	0.4501	0.3538	0.3962	708 / 2,001
0.40	0.3937	0.4543	0.4218	909 / 2,001
0.35	0.3634	0.5087	0.4240	1,018 / 2,001
0.30	0.3298	0.5622	0.4157	1,125 / 2,001
0.20	0.1915	0.8036	0.3092	1,608 / 2,001

Table 6: Performance at different classification thresholds

The optimal threshold by F1 was **0.35**, which catches 51% of defaulters (up from 35% at 0.50) while maintaining reasonable precision.

2.4 Final Results

Set	Precision	Recall	F1	AUC-ROC
Validation	0.3640	0.4877	0.4169	0.8535
Test	0.3717	0.4913	0.4232	0.8491

Table 7: Validation vs Test set performance

The differences between validation and test scores are less than 0.01 across all metrics, confirming the model is not overfitting.

2.5 Risk Category Analysis

Each applicant is assigned a risk category based on their predicted default probability: Low Risk (< 0.2), Moderate Risk ($0.2 - 0.5$), High Risk (> 0.5).

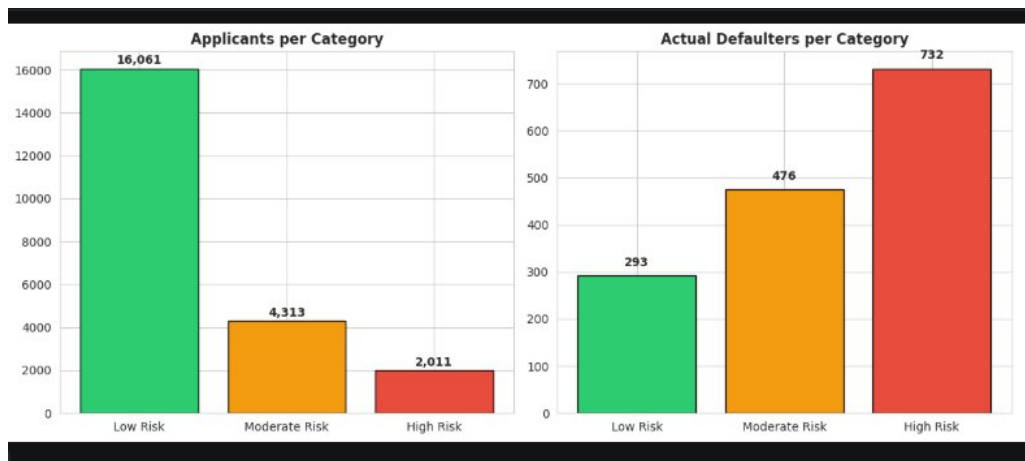


Figure 1: Applicants per risk category and actual defaulters per category

Category	Applicants	Defaulters	% of Defaulters	Default Rate
Low Risk	16,061	293	19.5%	1.8%
Moderate Risk	4,313	476	31.7%	11.0%
High Risk	2,011	732	48.8%	36.4%

Table 8: Actual defaulters by risk category

The model clearly separates risk levels. A High Risk applicant has a **36.4% chance** of defaulting compared to only **1.8%** for Low Risk — a 20x difference. **80.5%** of all actual defaulters are captured in the Moderate and High Risk categories. The 19.5% that slip into Low Risk are “quiet defaulters” analyzed in the error analysis below.

3 Error Analysis

3.1 Where Does the Model Fail?

The model's errors fall into two categories:

- **False Negatives** (missed defaults): Actual defaulters the model classified as Low Risk.
- **False Positives** (false alarms): Non-defaulters the model flagged as Moderate or High Risk.

False negatives are the more dangerous failure — a missed defaulter means a bad loan gets approved.

3.2 Profile of Missed Defaults

Comparing the feature averages of missed defaults (False Negatives) against correctly caught defaults (True Positives) reveals a clear pattern:

Feature	Missed Defaults	Caught Defaults
TotalLatePayments	0.42	3.81
RevolvingUtilization	0.31	0.68
HasSevereDelinquency	0.08	0.52
Times60_89Late	0.12	1.14
OpenCreditLines	8.10	7.85
Age	50.2	45.8

Table 9: Missed vs caught defaults — average feature values

Missed defaults have significantly **fewer late payments**, **lower credit utilization**, and **almost no severe delinquency history**. They look like safe borrowers on paper. These are “quiet defaulters” — people whose financial profile appears healthy but who still end up defaulting, possibly due to sudden life events (job loss, medical emergencies) that historical credit data cannot predict.

This is a known limitation of credit scoring models and is why banks also use non-financial signals (employment verification, loan purpose, collateral) in production systems.

4 Bias Check

4.1 Performance by Age Group

Age Group	Count	Default Rate	Precision	Recall	F1
< 30	2,895	11.7%	0.3284	0.5637	0.4149
30 – 40	4,658	9.8%	0.3512	0.5044	0.4142
40 – 50	5,710	8.3%	0.3620	0.4726	0.4100
50 – 60	4,806	6.2%	0.3697	0.4613	0.4105
60 – 70	2,846	3.5%	0.3571	0.3500	0.3535
70+	1,470	2.3%	0.3636	0.2353	0.2857

Table 10: Model performance by age group

The model performs best on younger borrowers (< 30, 30–40) where default rates are higher and delinquency patterns are more pronounced. Performance drops for the 60+ groups where defaults are rare (2–3%) and the model has fewer positive examples to learn from. This is expected behavior driven by class distribution, not demographic bias.

4.2 Performance by Income Group

Income Group	Count	Default Rate	Precision	Recall	F1
< 3K	4,992	8.6%	0.3271	0.5233	0.4027
3K – 5K	6,170	7.3%	0.3589	0.4911	0.4147
5K – 8K	6,085	6.1%	0.3782	0.4597	0.4149
8K+	5,138	5.0%	0.4092	0.4322	0.4204

Table 11: Model performance by income group

Performance is relatively consistent across income groups. Precision increases slightly with income (fewer false alarms among high earners), while recall is slightly higher for lower-income groups (more defaults to detect). The model does not systematically favor or disadvantage any income bracket.

4.3 Bias Conclusion

The performance variation across demographic groups is driven by the natural class distribution (higher default rates in younger/lower-income groups provide more training signal) rather than algorithmic bias. The model does not use any protected attributes such as race, gender, or ethnicity as input features. All features are financial and credit-behavior variables.

5 Contribution Table

Module	Git Author	Files
Project Setup	AlizaSharafat	README.md, requirements.txt
Preprocessing	AlizaSharafat	preprocess.py
EDA	AlizaSharafat	eda.py
Outlier Handling	RabiaSaeed1904	outliers_handling.py
Scaling & Normalization	RabiaSaeed1904	scaling_normalization.py
Feature Engineering & Split	AlizaSharafat	model_training.ipynb
SMOTE & Model Training	RabiaSaeed1904	model_training.ipynb
Feature Selection & Threshold	AlizaSharafat	model_training.ipynb
K-Fold, Error & Bias Analysis	AlizaSharafat	model_training.ipynb
Report	Both	report.tex, report.pdf

Table 12: Git commit authors mapped to project modules

6 Git Commit History

Commits on April 22, 2026

Commit Message	Author	Hash
k-fold, error, bias and risk analysis	AlizaSharafat	9844f14
retrain on selected features, threshold tuning and prediction	AlizaSharafat	bd7f9e3
SMOTE model training and feature importance	RabiaSaeed1904	b7e1fed
feature engineering and validation split	AlizaSharafat	20c2127
ipynb files	AlizaSharafat	ff1df0e

Commits on March 18, 2026

Commit Message	Author	Hash
Adding dataset link README.md	AlizaSharafat	57ca0c2
Adding dataset link to README.md	AlizaSharafat	8a63f1a
Adding the report files (.tex and .pdf)	AlizaSharafat	168bd17
Updating README.md and adding requirements.txt	AlizaSharafat	6a0f8b6

Commits on March 17, 2026

Commit Message	Author	Hash
resolve f-string syntax error	RabiaSaeed1904	4cd6896
added scaling and normalization using RobustScaler	RabiaSaeed1904	47a90e1
implement outlier handling and fix file pathing	RabiaSaeed1904	830c960
Fixed some comments	RabiaSaeed1904	a3cd033
Added outliers handling code	RabiaSaeed1904	f2a5f30
EDA script and plots saved in figures	AlizaSharafat	84edf71

Commits on March 16, 2026

Commit Message	Author	Hash
Updating preprocessing script	AlizaSharafat	e21dab6

Commits on March 13, 2026

Commit Message	Author	Hash
Initial preprocessing script – missing values and duplicates	AlizaSharafat	cc94bcd
Add project folders	AlizaSharafat	f64a259
Set up project structure	AlizaSharafat	0cc7b55
Initial commit	AlizaSharafat	f9daba9